

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль №2

Вариант Б-5

ДИСЦИПЛИНА: «Парадигмы и конструкции языков программирования»

Выполнил: студент гр. ИУ5-32Б _____ (Голуб А.С.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю.Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

2026 г.

Описание задания

Рубежный контроль №2. Вариант Б-5. Предметная область: «Музыкант – Оркестр».

1. Выполнить рефакторинг программы РК №1, чтобы её запросы можно было тестировать независимо от консольного вывода.
2. Разработать три модульных теста с применением TDD-фреймворка.

Рефакторинг

Классы Musician, Orchestra, MusicianOrchestra и исходный набор данных сохранены. Функции query_b1(), query_b2(), query_b3() принимают коллекции объектов как аргументы и возвращают результаты без печати в консоль. Файл main.py отвечает только за получение данных и вывод.

Каждая функция соответствует одному запросу варианта Б: сортировка музыкантов, подсчёт количества музыкантов по оркестрам, отбор фамилий на «ов» по связи многие-ко-многим.

Для модульного тестирования выбран pytest. Для каждого запроса создан отдельный тест с заранее известным ожидаемым результатом. Цикл TDD: Red (проверки не проходят для незавершённых функций), Green (реализация проходит тесты), Refactor (разделение логики и вывода при сохранении результата).

Текст программы

rk.py

```
class Musician:
    def __init__(self, id, surname, experience, orchestra_id):
        self.id = id
        self.surname = surname
        self.experience = experience
        self.orchestra_id = orchestra_id


class Orchestra:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class MusicianOrchestra:
    def __init__(self, musician_id, orchestra_id):
        self.musician_id = musician_id
        self.orchestra_id = orchestra_id


def make_data():
    orchestras = [
        Orchestra(1, "Симфонический оркестр"),
```

```
        Orchestra(2, "Камерный оркестр"),
        Orchestra(3, "Молодёжный оркестр"),
        Orchestra(4, "Джазовый оркестр"),
    ]
    musicians = [
        Musician(1, "Иванов", 10, 1),
        Musician(2, "Петров", 7, 1),
        Musician(3, "Сидоров", 12, 2),
        Musician(4, "Смирнова", 5, 2),
        Musician(5, "Орлова", 3, 3),
    ]
    links = [
        MusicianOrchestra(1, 1),
        MusicianOrchestra(1, 2),
        MusicianOrchestra(1, 4),
        MusicianOrchestra(2, 1),
        MusicianOrchestra(2, 3),
        MusicianOrchestra(3, 2),
        MusicianOrchestra(4, 3),
        MusicianOrchestra(5, 4),
    ]
    return musicians, orchestras, links


def query_b1(musicians, orchestras):
    pairs = [
        (m.surname, o.name)
        for m in musicians
        for o in orchestras
        if m.orchestra_id == o.id
    ]
    return sorted(pairs, key=lambda pair: pair[0].casefold())


def query_b2(musicians, orchestras):
    counts = [
        (o.name, sum(m.orchestra_id == o.id for m in musicians))
        for o in orchestras
    ]
    return sorted(counts, key=lambda pair: (-pair[1], pair[0].casefold()))


def query_b3(musicians, orchestras, links):
    pairs = [
        (m.surname, o.name)
        for link in links
        for m in musicians
        for o in orchestras
        if link.musician_id == m.id
        and link.orchestra_id == o.id
        and m.surname.casefold().endswith("ов")
    ]
    return sorted(pairs)
```

test_rk.py

```
from rk import make_data, query_b1, query_b2, query_b3


def test_b1_sorted_musicians():
    musicians, orchestras, _ = make_data()
    assert query_b1(musicians, orchestras) == [
        ("Иванов", "Симфонический оркестр"),
```

```
        ("Орлова", "Молодёжный оркестр"),
        ("Петров", "Симфонический оркестр"),
        ("Сидоров", "Камерный оркестр"),
        ("Смирнова", "Камерный оркестр"),
    ]


def test_b2_orchestra_counts():
    musicians, orchestras, _ = make_data()
    assert query_b2(musicians, orchestras) == [
        ("Камерный оркестр", 2),
        ("Симфонический оркестр", 2),
        ("Молодёжный оркестр", 1),
        ("Джазовый оркестр", 0),
    ]


def test_b3_musicians_ending_ov():
    musicians, orchestras, links = make_data()
    assert query_b3(musicians, orchestras, links) == [
        ("Иванов", "Джазовый оркестр"),
        ("Иванов", "Камерный оркестр"),
        ("Иванов", "Симфонический оркестр"),
        ("Петров", "Молодёжный оркестр"),
        ("Петров", "Симфонический оркестр"),
        ("Сидоров", "Камерный оркестр"),
    ]
```

main.py

```
from rk import make_data, query_b1, query_b2, query_b3


def main():
    musicians, orchestras, links = make_data()

    print("Задание Б1. Музыканты и оркестры (по фамилиям):")
    for surname, orchestra in query_b1(musicians, orchestras):
        print(f"{surname} - {orchestra}")

    print("\nЗадание Б2. Количество музыкантов в каждом оркестре:")
    for orchestra, count in query_b2(musicians, orchestras):
        print(f"{orchestra}: {count}")

    print("\nЗадание Б3. Фамилии на «ов» (многие-ко-многим):")
    for surname, orchestra in query_b3(musicians, orchestras, links):
        print(f"{surname} - {orchestra}")


if __name__ == "__main__":
    main()
```

Примеры выполнения работ

Запуск программы демонстрирует, что после рефакторинга сохранена функциональность РК №1: результаты всех трёх запросов совпадают с исходными.

```
PS D:\Users\G_lean_O\source\repos\PCPL\RK2> py main.py
Задание Б1. Музыканты и оркестры (по фамилиям):
Иванов — Симфонический оркестр
Орлова — Молодёжный оркестр
Петров — Симфонический оркестр
Сидоров — Камерный оркестр
Смирнова — Камерный оркестр

Задание Б2. Количество музыкантов в каждом оркестре:
Камерный оркестр: 2
Симфонический оркестр: 2
Молодёжный оркестр: 1
Джазовый оркестр: 0

Задание Б3. Фамилии на «ов» (многие-ко-многим):
Иванов — Джазовый оркестр
Иванов — Камерный оркестр
Иванов — Симфонический оркестр
Петров — Молодёжный оркестр
Петров — Симфонический оркестр
Сидоров — Камерный оркестр
```

Команда python -m pytest -q обнаружила три теста: test_b1_sorted_musicians, test_b2_orchestra_counts, test_b3_musicians_ending_ov. Все тесты завершились успешно (3 passed).

